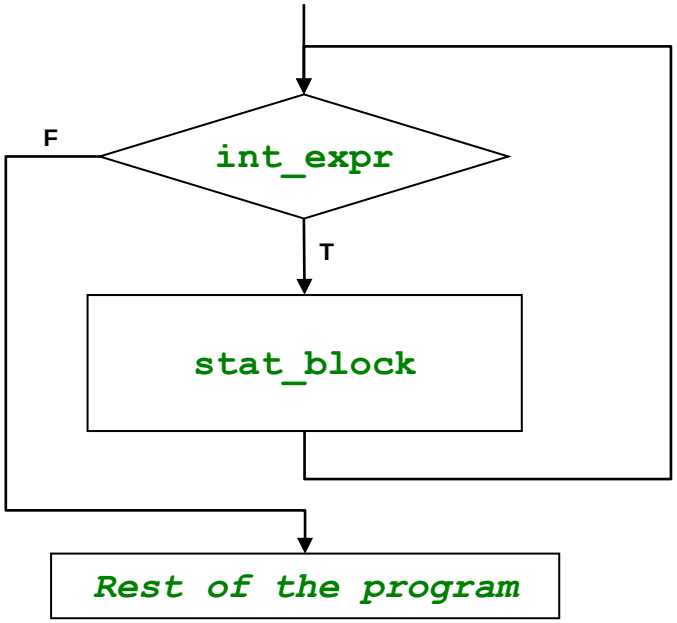
While loop

- Use while loop if the statement block should be executed as long as a condition holds.
- Syntax:

```
while (int_expr)
    stat_block
```



while: Example 1

Find the average of a sequence of integers terminated with a negative value.

```
int sum=0, n, count=0;
float avg;

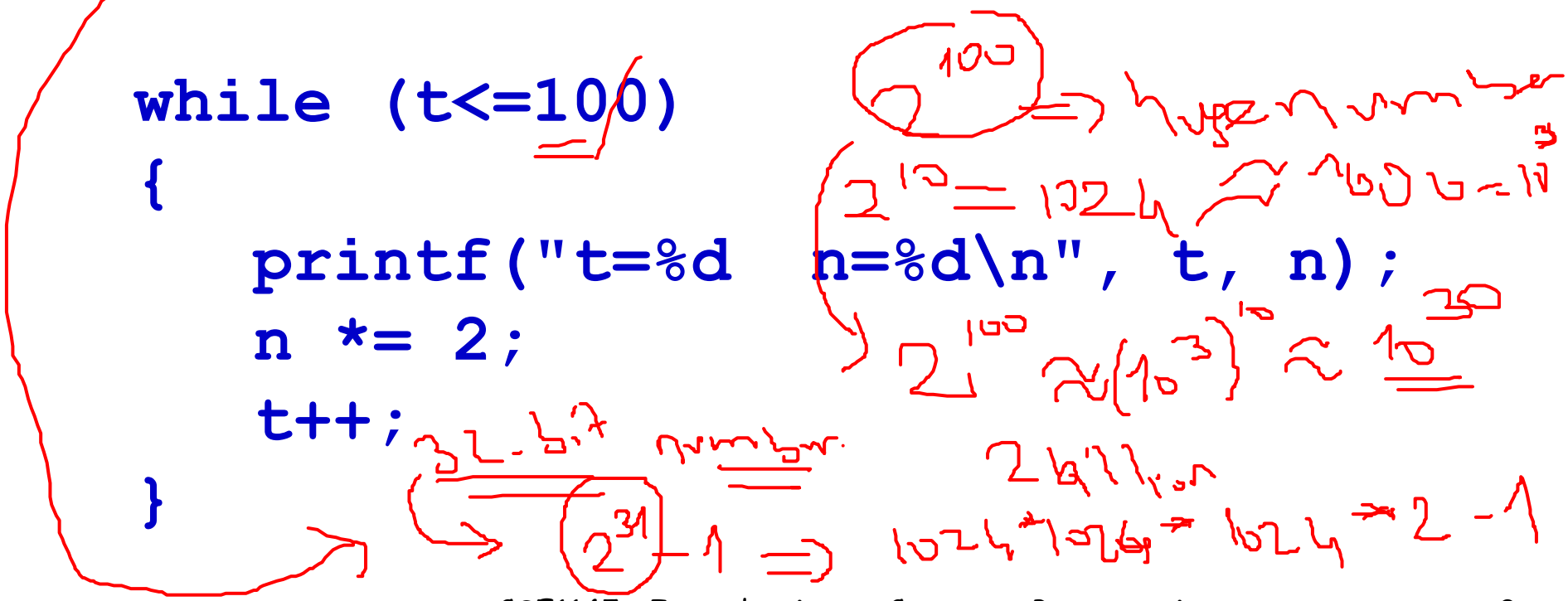
scanf("%d", &n);
while (n>=0)
{
    sum += n;
    count++;
    scanf("%d", &n);
}
avg = (count)?(float)sum/count:0;
```

while: Example 2

Consider a type of cell that reproduces by mitosis. Assume each cell divides into two cells every second. Display the number of cells after each second for 100 seconds. Start with one cell.

```
int n=1, t=0;

while (t<=100)
{
    printf("t=%d  n=%d\n", t, n);
    n *= 2;
    t++;
}
```



while: Example 3

Assume the user enters a sequence of 1s and 0s. Any other character marks the end of input. Take 1's complement of the input.

```
char ch;

ch=getchar();
while ((ch=='0') || (ch=='1'))
{
    printf("%d", !(ch-'0'));
    ch=getchar();
}
```

while: Example 4

What is the output of the following code segment?

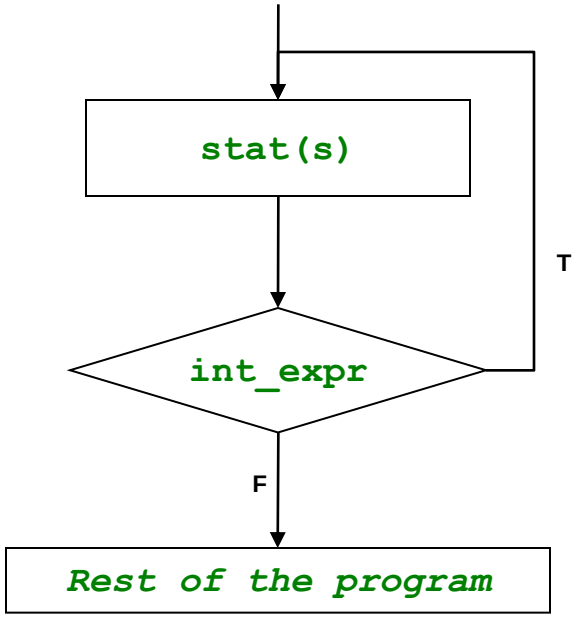
```
int i;


while (++i<10);
    printf("%d;", i-=2);
printf("{%d}", i);
```

Do-while loop

- Similar to while loop.
  - Only difference: Condition is checked after the execution of every iteration.
- Syntax:

```
do
{
    stat(s)
}
while (int_expr);
```



do-while: Example 1

Solve the previous example, this time using do-while.

Find the average of a sequence of integers terminated with a negative value.

```
int sum=0, n, count=0;
float avg;

do
{
   scanf("%d", &n);
   if (n>=0)
   {
      sum += n;
      count++;
   }
} while (n>=0);
avg = (count)?(float)sum/count:0;
```

Note that we repeated the condition in the if statement, so it was not a good idea to solve this problem using do-while.

do-while: Example 2

Find the sum of the integers as long as the user wants to enter input.

```
int sum=0, n;
char ch;

do
{
   scanf("%d", &n);
   sum += n;
   printf("Do you want to exit? (y/n) ");
   ch=getchar();
} while ((ch!='y')&& (ch!='Y'));
```

break statement

- It is possible to terminate a loop prematurely.
- Remember the **`break`** statement we discussed before.
  - **`break`** breaks the innermost loop or switch statement.

break: Example

Read 100 integers and find their product. However, if one of the numbers is non-positive, stop input.

```
long int mul=1;
int i, num;

for (i=0; i<100; i++)
{
    scanf("%d", &num);
    if (num<=0)
        break;
    mul *= num;
}
printf("%ld", mul);
```

continue statement

- It is possible to skip the rest of an iteration and continue with the next iteration (if any).
  - In the for loop, **`continue`** jumps to the final statement.
  - In the while and do-while loops, **`continue`** jumps to the condition expression.

continue: Example 1

Consider the following two code segments.

Assume input is: `5 -4 3 2 -5 8 9 1`

```
sum = i = 0;
while (i < 6)
{
   scanf("%d",&no);
   if (no <= 0)
      continue;
   sum += no;
   i++;
}
/* sum becomes ? */
```

```
sum = 0;
for (i = 0; i < 6; i++)
{
   scanf("%d",&no);
   if (no <= 0)
      continue;
   sum += no;
}

/* sum becomes ? */
```

continue: Example 2

Find the number of passengers/car for every flat in a building with 40 flats.

```
for (i=0; i<40; i++)
{
    scanf("%d", &no_cars);
    if (no_cars==0)
        continue;
    scanf("%d", &no_residents);
    printf("%f\n",
            (float)no_residents/no_cars);
}
```

Example: Nested loops

- You can nest the loops as you do with if statements.

Nested loops: Example 1

Draw an isosceles triangle using '*' character. Number of lines is read as input.

```
#include <stdio.h>

int main()
{
   int line, i, j;

   printf("Enter the height :");
   scanf("%d",&line);
   for (i = 1; i <= line; i++)
   {
      for (j = 0; j < line - i; j++)
         printf(" ");
      for (j = 0; j < i * 2 - 1; j++)
         printf("*");
      printf("\n");
   }
   return 0;
}
```

Nested loops: Example 2

Read a whole number char-by-char and assign it to an integer variable. Blank terminates input. Ignore any other character.

```
int number=0;
char ch;

while (1)
{
   do {
      ch=getchar();
   } while ((ch!=' ') &&((ch<'0')||('9'<ch));
   if (ch==' ')
      break;
   number = (number*10)+(ch-'0');
}
```

Example

- Read an integer and print its digits in reverse order.

```
#include <stdio.h>

int main()
{
    int num, digit;
    scanf("%d",&num);
    while (num)
    {   digit=num%10;
        num /= 10;
        printf("%d", digit);
    }
    return 0;
}
```

Example

- Read a float. Exchange the decimal and whole parts. Eg: If the input is 35.794678, the output should be 794678.35

```
#include <stdio.h>

int main()
{  float num, dec, whole;

   scanf("%f",&num);
   whole = (int) num;
   dec = num-whole;
   while (whole>1)
      whole /= 10;
   while (dec != (int)dec)
      dec *= 10;
   num = dec+whole;
   printf("%f", num);
   return 0;
}
```

You think this should work, but it won't!!!

The problem is due to the representation of floating point numbers. Try debugging it.

Example

- Solve the same question by considering only the first four digits after the decimal point to be significant. Eg: If the input is 35.794678, the output should be 7946.35.

```
#include <stdio.h>

int main()
{   float num, dec, whole;
    int i;

    scanf("%f",&num);
    whole = (int) num;
    dec = num-whole;
    while (whole>1)
       whole /= 10;
    for (i=0; i<4; i++)
      dec *= 10;
    num = (int)dec+whole;
    printf("%f", num);
    getchar();
    return 0;
}
```